

TECHNICAL WHITE PAPER

resume-forge

Local-First, LLM-Powered Resume Tailoring
with Deterministic ATS Scoring

Core Library ■ Web Application ■
MCP Server ■ Command-Line Interface

Version	1.0 — Feature-Complete Release
Date	July 2026
Classification	Internal / Project Documentation
Repository	github.com/vishalVasanthakumarPoornima/ Resume-Tailor-ATS-Scorer

Contents

Executive Summary	2
1 Introduction and Problem Statement	3
1.1 The Applicant Tracking System Problem	3
1.2 Why Manual Tailoring Fails to Scale	3
1.3 Design Goals	3
2 Solution Overview	4
2.1 End-to-End Flow	4
3 System Architecture	5
3.1 Four Interfaces, One Core	5
3.2 Layer Responsibilities	5
4 Pipeline Deep Dive	6
4.1 Stage 1 — Resume Ingestion	6
4.2 Stage 2 — Job Extraction	6
4.3 Stage 3 — Tailoring	6
4.4 Stage 4 — L ^A T _E X Rendering	6
4.5 Stage 5 — PDF Compilation	6
4.6 Stage 6 — ATS Scoring	6
4.7 Stage 7 — Optimization Loop	7
4.8 Stage 8 — Optional Cover Letter	7
5 Multi-Provider Inference Strategy	8
5.1 The Problem With Binding to One Provider	8
5.2 Supported Backends	8
5.3 Selection Logic	8
5.4 Structured Output on Constrained Local Models	8
6 Data Integrity: The No-Fabrication Guarantee	10
6.1 Why a Prompt Alone Is Not Enough	10
6.2 Layer 1 — The Explicit Rule	10
6.3 Layer 2 — The Structural Guard	10
6.4 Layer 3 — Truthful Keyword Backfill	10
7 ATS Scoring Methodology	11
7.1 Design Philosophy	11
7.2 Weighted Dimensions	11
7.3 Output	11
8 Resume Rendering and the One-Page Guarantee	12
8.1 Template Design	12
8.2 Guaranteeing a Single, Single-Sided Page	12
9 Security and Privacy	13
9.1 Data Handling	13
9.2 File System Boundaries	13
10 Quality Assurance and Testing	14
10.1 Test Suite	14
10.2 Continuous Integration	14
10.3 Manual End-to-End Verification	14
11 Interfaces and Integration	15
11.1 Command-Line Interface	15
11.2 Python API	15
11.3 MCP Server — Agentic Integration	15
11.4 Web Application	15

Executive Summary

resume-forge is a full-stack system that tailors a candidate’s resume to a specific job posting, scores the result against a deterministic, transparent model of an Applicant Tracking System (ATS), and iterates automatically until a target score is met — all while mechanically guaranteeing that no employer, degree, certification, or skill is ever invented. The system is delivered as four cooperating interfaces built on one shared core: a Python library, a web application, a Model Context Protocol (MCP) server for agentic integration, and a command-line tool.

The defining engineering decision in this project is **provider independence for inference**. Rather than binding the product to a single LLM vendor, resume-forge implements a pluggable backend layer that auto-detects the best available option at runtime: a free-tier cloud model (z.ai/GLM, Google Gemini, Groq, or Puter) when a credential is present, or a fully local Ollama model with zero external dependency otherwise. This yields a system that runs identically whether the operator has no API budget at all, a free-tier key, or an enterprise subscription to Anthropic’s Claude — with cloud inference reducing per-run latency from minutes to seconds.

4	8	94	7
Product Interfaces	Pipeline Stages	Automated Tests	LLM Backends

This document describes the problem resume-forge solves, its architecture and data-integrity guarantees, its scoring methodology, its multi-provider inference strategy, its security and privacy posture, and the quality assurance process behind it.

1 Introduction and Problem Statement

1.1 The Applicant Tracking System Problem

The overwhelming majority of mid-size and large employers filter inbound resumes through an Applicant Tracking System before a human ever sees them. These systems parse résumé text, match it against keyword and skill taxonomies extracted from the job requisition, and rank or reject candidates algorithmically. A resume that is a strong substantive match for a role can still be filtered out for entirely mechanical reasons: a non-standard section header, a graphics-heavy template that fails text extraction, a multi-column layout that scrambles reading order, or simply an absence of the specific terminology the posting uses.

1.2 Why Manual Tailoring Fails to Scale

The standard advice — “tailor your resume to every job” — is sound but expensive. A candidate applying to twenty roles in a search would need to manually re-read each posting, identify its priority keywords, and rewrite bullet content to surface the right prior experience in the right language, without ever crossing the line into embellishment or fabrication. This is slow, inconsistent under fatigue, and difficult to self-audit: a candidate under pressure to hit a keyword target has a natural incentive to stretch the truth, whether intentionally or not.

1.3 Design Goals

resume-forge was built against four non-negotiable requirements:

1. **Never fabricate.** Tailoring must be limited to emphasis, reordering, and rewording of facts already present in the candidate’s master resume. This must be enforced in code, not merely requested in a prompt.
2. **Score against a real, inspectable model of ATS behavior**, not a black box — so the candidate can see exactly why a resume did or did not clear a threshold and what to do about it.
3. **Run without a paid API dependency**, so cost is never a barrier to iterating on a job search, while still allowing an upgrade path to faster or higher-quality inference when available.
4. **Be usable four ways:** interactively via a browser, programmatically as a Python library, as a tool an AI agent can call (MCP), and as a scriptable CLI — so it composes with other tooling rather than being a dead-end application.

2 Solution Overview

resume-forge accepts two inputs — an existing resume (PDF, DOCX, $\text{\LaTeX}$, or plain text) and a job posting (a URL or pasted text) — and produces a tailored, compiled, single-page PDF resume together with a numeric ATS compatibility score, a breakdown of that score by dimension, and a list of concrete, truthful improvements still available. An optional grounded cover letter can be generated in the same pass.

What Makes This Different

- **Structural fabrication guard.** Tailored output is diffed against the candidate’s verified master profile using fuzzy entity matching; any employer or institution the model invents is discarded, and any the model omits is *automatically restored* from the source of truth.
- **Deterministic scorer.** The ATS score is computed by plain Python — regular expressions, section detection, and weighted keyword coverage — not by asking an LLM to grade its own work.
- **One-page guarantee.** The $\text{\LaTeX}$ rendering pipeline enforces single-page, single-sided output through progressive spacing compaction rather than truncating content.
- **Provider-agnostic inference.** The same pipeline runs unmodified on a free cloud API, a paid frontier model, or a fully offline local model.

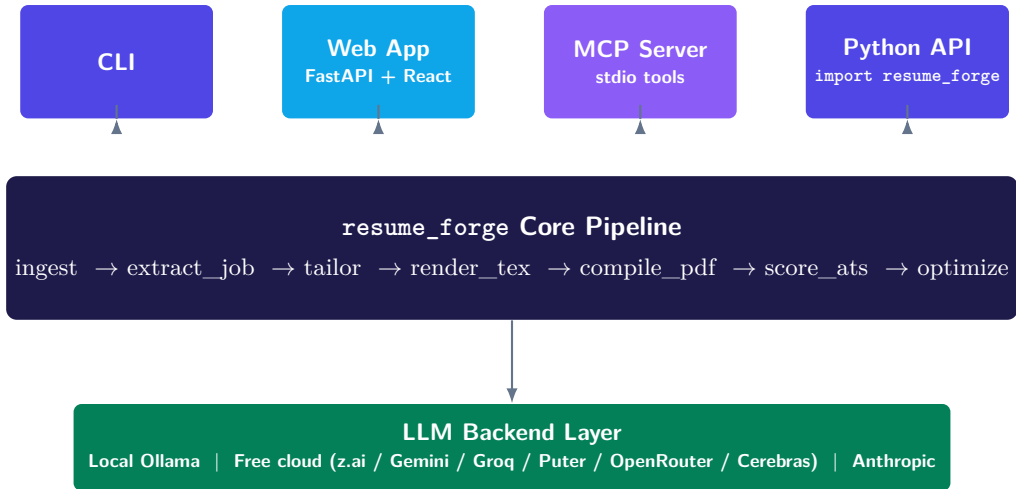
2.1 End-to-End Flow

1. The candidate’s master resume is parsed once into a structured profile (contact, summary, skills, experience, projects, education, certifications) and cached by content hash, so repeated runs against the same resume skip re-parsing entirely.
2. The job posting is fetched (or accepted as pasted text) and extracted into a structured requirements object: title, company, required skills, preferred skills, keywords, and responsibilities.
3. The tailoring stage rewrites and reorders the master profile’s content to match the posting’s language, under a hard no-fabrication instruction enforced both in the prompt and in a post-processing guard.
4. The tailored content is rendered into an ATS-friendly $\text{\LaTeX}$ template and compiled to PDF.
5. The compiled PDF is scored by the local, deterministic ATS scorer.
6. If the score is below the target, the scorer’s specific feedback — missing keywords, weak subscores — is fed back into the tailoring stage, and steps 3–5 repeat, up to a configurable iteration cap.
7. The best-scoring result across all iterations is returned, together with the full score report, even if the target was never reached.

3 System Architecture

3.1 Four Interfaces, One Core

resume-forge is deliberately layered so that the tailoring logic is written exactly once and consumed four different ways.



3.2 Layer Responsibilities

Layer	Responsibility
Core library	Every pipeline stage as an independently testable, pure(ish) Python function with Pydantic-validated inputs and outputs. No interface-specific logic lives here.
Web application	FastAPI backend running jobs on a background thread with live per-stage progress; a React + Vite + Tailwind frontend with animated, branded UI components. Ships in a single-process production mode.
MCP server	Exposes <code>tailor_resume</code> , <code>extract_job_posting</code> , <code>score_resume</code> , and <code>ingest_resume</code> as Model Context Protocol tools over stdio, so any MCP-aware agent (including other Claude-based projects) can call the pipeline directly.
CLI	A thin <code>argparse</code> wrapper (<code>resume-forge</code>) for scripted or terminal use, supporting stdin piping for job descriptions.

4 Pipeline Deep Dive

Every stage below is an independently importable, independently tested function in the `resume_forge` package.

4.1 Stage 1 — Resume Ingestion

The master resume (PDF via `pdfplumber`, DOCX via `python-docx`, or plain text/ $\text{\LaTeX}$) is extracted to text and parsed once by the LLM into a structured `MasterProfile`: contact information, summary, categorized skills, work experience with dated bullets, projects, education, and certifications. The result is cached to disk keyed by a hash of the file's bytes, so identical resumes are never re-parsed.

Reliability guard. Small or quantized language models occasionally return a schema-valid but empty parse (Section 6.3 explains why). `resume-forge` detects a suspiciously empty result against a non-trivial resume, retries once with a corrective prompt, and — on Ollama specifically — escalates to the strongest locally installed model as a last resort before failing loudly. It never silently returns an empty resume.

4.2 Stage 2 — Job Extraction

A job posting URL is fetched with a single polite `httpx` request and its main content extracted with `trafilatura` (BeautifulSoup as fallback). If that fails — common on job boards that block automated access — `resume-forge` falls back to an optional headless Chromium render (Playwright) for JavaScript-rendered postings, and finally to directly-pasted text. The result is parsed into required skills, preferred skills, general keywords, and responsibilities.

4.3 Stage 3 — Tailoring

The master profile and job requirements are given to the LLM under an explicit hard rule (reproduced in full in Section 6) that tailoring means only emphasis, reordering, and rewording of existing facts. Output is a `TailoredResume` of the same shape as the master profile.

4.4 Stage 4 — $\text{\LaTeX}$ Rendering

Tailored content is rendered into a Jinja2-templated $\text{\LaTeX}$ document using `\VAR{}`/`\BLOCK{}` delimiters (chosen so the delimiters themselves are valid $\text{\LaTeX}$ and do not collide with document syntax). All user-derived text passes through a dedicated escaping filter that handles both the seven $\text{\LaTeX}$ special characters and Unicode punctuation (em/en dashes, curly quotes, ellipses) that would otherwise silently disappear under T1 font encoding.

4.5 Stage 5 — PDF Compilation

The $\text{\LaTeX}$ source is compiled with `tectonic`, a self-contained $\text{\TeX}$ engine that fetches packages on demand, with `pdflatex` as a fallback if `tectonic` is unavailable. Compiler errors are parsed and surfaced as a focused log excerpt rather than a raw dump.

4.6 Stage 6 — ATS Scoring

The compiled PDF's text is extracted and scored by a fully local, deterministic model of ATS behavior. Full methodology in Section 7.

4.7 Stage 7 — Optimization Loop

If the score is below target, the scorer's missing keywords and weakest subscores are fed back into Stage 3 as structured feedback, and Stages 3–6 repeat up to a configurable iteration cap (default 5). The no-fabrication guarantee holds on every iteration — a keyword the master profile cannot truthfully support will simply remain missing, and the final report says so explicitly rather than pretending otherwise.

4.8 Stage 8 — Optional Cover Letter

When requested, a grounded cover letter is generated under the same fabrication contract: the model writes body paragraphs only, while the salutation, date, and signature are assembled from verified data. Placeholder tokens and stray sign-offs are stripped programmatically as a second line of defense.

5 Multi-Provider Inference Strategy

5.1 The Problem With Binding to One Provider

Any tool that hard-codes a single LLM vendor inherits that vendor's cost structure, rate limits, and availability as its own. `resume-forge` instead treats the LLM as a swappable component behind a two-method protocol (`parse(system, prompt, output_type) -> output_type`), with every backend returning schema-validated, Pydantic-typed output.

5.2 Supported Backends

Backend	Cost	Latency	Notes
z.ai (GLM)	Free tier	Seconds	Default auto-detected cloud backend; <code>glm-4.5-flash</code> / <code>glm-4.7-flash</code> genuinely free, not trial credit
Google Gemini	Free tier	Seconds	Largest free daily quota of the supported providers
Groq	Free tier	Seconds	Fastest raw inference speed observed
Puter	Free (token)	Seconds	Resells GLM and other models with no z.ai account required; one free Puter token
OpenRouter / Cerebras	Free tier	Seconds	Additional free-tier options with presets
Generic OpenAI-compatible	Varies	Varies	Escape hatch for any compatible endpoint, including keyless self-hosted servers
Ollama (local)	Free	Tens of seconds	Fully offline; zero external dependency; used automatically when no cloud credential is present
Anthropic (Claude)	Paid	Seconds	Opt-in only; never auto-selected, since it is the only backend with a direct cost

5.3 Selection Logic

At startup, `resume-forge` resolves the active backend in strict priority order: an explicit `-backend` flag or `RESUME_FORGE_LLM_BACKEND` environment variable wins outright; otherwise the system scans, in order, for a present API key among the free cloud providers and auto-selects the first one found; if none is present, it falls back to a local Ollama model with no configuration required at all. Anthropic is deliberately excluded from auto-detection so that a stray credential in the environment never causes an unexpected charge.

Local-first fallback. Because the local Ollama path requires zero credentials and zero network access, `resume-forge` is guaranteed to run in a fully air-gapped environment. Cloud backends are a latency optimization layered on top of that guarantee, never a requirement.

5.4 Structured Output on Constrained Local Models

A significant engineering finding from this project concerns Ollama's grammar-constrained JSON decoding: it emits object keys in *schema declaration order* and can only skip optional keys, never revisit them. A source document (such as a resume) presenting information in a different order than the schema — for example, listing education before work experience — caused the model to silently omit every field “before” the point it began writing, producing a schema-valid but substantively empty parse. The fix, now applied unconditionally, marks every property in every object of the JSON schema as required before it is sent to the

model, eliminating the entire failure class. This was verified by side-by-side testing on the same malformed input before and after the fix.

6 Data Integrity: The No-Fabrication Guarantee

6.1 Why a Prompt Alone Is Not Enough

Instructing a model not to fabricate is necessary but not sufficient — language models can still drift, omit, or subtly reword a fact into something false, especially under a smaller or quantized model. resume-forge treats this as a systems problem, not a prompting problem, and enforces the guarantee at three independent layers.

6.2 Layer 1 — The Explicit Rule

Every tailoring and cover-letter prompt embeds the following instruction verbatim:

“Only use facts present in the master profile. NEVER invent or alter employers, job titles, employment dates, degrees, institutions, GPAs, certifications, project names, or metrics. Every number in a bullet must already exist in the master profile. Tailoring means emphasis, reordering, rewording, and keyword alignment — NOT adding experience the candidate does not have. If the job asks for a skill the candidate lacks, simply do not mention it; do not imply familiarity.”

6.3 Layer 2 — The Structural Guard

After every tailoring call, the output is programmatically diffed against the master profile using fuzzy entity matching (canonicalized string comparison plus similarity scoring) that tolerates harmless cosmetic drift — “Perficient, Inc.” matching “Perficient” — while still catching genuine fabrication:

- Any employer, project, or certification in the tailored output that does not fuzzy-match an entry in the master profile is **removed**.
- Any employer or education entry present in the master profile but **omitted** by the model is **restored** — tailoring can structurally never delete a candidate’s employment history.
- Education entries are copied verbatim from the master profile without exception; rewording a degree (for example, rephrasing an in-progress program as completed) is a correctness risk with no offsetting benefit.
- Contact information is always taken from the verified profile, never from model output, with a regex-based backfill recovering fields (email, phone, LinkedIn, GitHub) that a smaller model sometimes drops during parsing.

6.4 Layer 3 — Truthful Keyword Backfill

Conversely, if the model over-trims a skill that genuinely exists in the master profile and is also requested by the job posting, it is restored into the tailored skills section. This ensures the optimization loop can close truthful keyword gaps without the fabrication guard fighting against legitimate emphasis.

Net Effect

The tailored resume is provably a strict subset of facts already present in the candidate’s verified master profile, rearranged and reworded for relevance — with any additions to, or omissions from, either direction reconciled automatically against the source of truth.

7 ATS Scoring Methodology

7.1 Design Philosophy

Rather than asking an LLM to self-grade the resume it just wrote — an approach vulnerable to reward hacking and inconsistent grading — resume-forge scores the *compiled PDF's extracted text* with plain, deterministic Python. This mirrors how a real ATS operates: it never sees the source $\text{\LaTeX}$ or the model's intent, only what a PDF text extractor can pull out of the finished document.

7.2 Weighted Dimensions

Dimension	Weight	What It Measures
Keyword coverage	40	Weighted match rate against the job's required and preferred skills and general keywords; required skills weigh double preferred/general terms
Parseability	15	Clean text extraction from the PDF — penalizes embedded-font artifacts, encoding replacement characters, and unusually low text density per page
Standard sections	15	Presence of conventional headers (Experience, Education, Skills) that ATS parsers are tuned to recognize
Bullet quality	15	Fraction of content lines beginning with a strong action verb and fraction carrying a quantified result (a number, percentage, or dollar figure)
Contact information	10	Machine-detectable email address and phone number
Length	5	Page count and total word count within a reasonable range for the format
Total	100	

7.3 Output

Every scoring call returns the overall score, the six subscores against their maxima, the specific keywords still missing (required-weight keywords sorted first), and a list of actionable, plain-English suggestions — all of which feed the optimization loop's next tailoring attempt.

8 Resume Rendering and the One-Page Guarantee

8.1 Template Design

The L^AT_EX template is deliberately conservative for ATS compatibility: single column, standard fonts, no graphics, no tables used for content layout (only for genuinely tabular data), and real selectable text throughout. This is the same category of layout choice that most professional ATS-optimization guidance recommends, applied structurally rather than left to chance.

8.2 Guaranteeing a Single, Single-Sided Page

Rather than truncating content to force a page break — which risks silently dropping a qualification — the renderer applies a sequence of *progressively compact* spacing presets (margin, line spacing, section spacing, and bullet spacing) and re-compiles until the result fits one page, only reducing content as a last resort and never removing a full section. This preserves substance while producing the tight, professional density that both human recruiters and ATS parsers expect from a one-page resume.

9 Security and Privacy

No inbound network exposure. The web application binds to 127.0.0.1 by default and is intended for local, single-user use. Job state is held in-process; nothing is persisted to an external service.

9.1 Data Handling

- The candidate's resume never leaves the machine except as the prompt payload sent to whichever LLM backend is active — and when that backend is local Ollama, it never leaves the machine at all.
- API credentials are read from a local, git-ignored `.env` file and are never logged, echoed, or transmitted anywhere other than the configured provider's own API endpoint.
- Job-posting scraping performs a single polite HTTP request per attempt with a standard browser user agent — no login automation, no CAPTCHA or bot-wall evasion, and no retry hammering. Sites whose Terms of Service prohibit automated access (several major job boards among them) are expected to block the request; the documented and supported fallback is for the user to paste the posting text directly.

9.2 File System Boundaries

All model-supplied file paths (in the text-editor-style tool surfaces used during development and testing) are canonicalized and verified to remain within the intended project root before any read or write, rejecting path traversal via `..` sequences, symlinks, or encoded variants.

10 Quality Assurance and Testing

10.1 Test Suite

resume-forge ships with a fully offline, fully mocked automated test suite — no network access and no live LLM calls are required to run it. As of this release, **94 tests pass**, covering:

- ATS scorer logic and edge cases
- LaTeX special-character and Unicode escaping
- Job-description URL-fallback and browser-fallback paths
- The structural no-fabrication guard, including fuzzy-match and restoration behavior
- The truthful-skill backfill guard
- Ingest caching and cache invalidation
- Cover-letter sanitation (placeholder and sign-off stripping)
- Every OpenAI-compatible backend preset, including keyless-endpoint support
- Ollama structured-output retry and error-message behavior
- The full optimize loop under stubbed pipeline stages

10.2 Continuous Integration

A GitHub Actions workflow runs on every push and pull request: the Python test suite via `uv`, and a TypeScript type-check plus production build of the frontend, so a regression in either half of the stack is caught before merge.

10.3 Manual End-to-End Verification

Beyond the automated suite, this release was verified against real inputs at every layer: a genuine multi-section resume run through the full pipeline via the CLI, the same flow driven through the live web application and polled to completion, the MCP server exercised over a real stdio client session with an actual tool call, and the production static build served directly from the FastAPI backend.

11 Interfaces and Integration

11.1 Command-Line Interface

```
resume-forge -job <url|-> -resume <path> -out <dir> \
-target 80 -cover-letter -backend zai
```

11.2 Python API

Every pipeline stage is independently importable, enabling composition with other Python systems without going through a subprocess or HTTP boundary:

```
from resume_forge import forge
result = forge(job_url_or_text, resume_path, out_dir, target_score=80)
```

11.3 MCP Server — Agentic Integration

This is the interface most relevant to combining resume-forge with other projects. Any Model Context Protocol client — including Claude Code, Claude Desktop, or a custom agent in an unrelated codebase — can register resume-forge as a tool provider over stdio and call `tailor_resume`, `extract_job_posting`, `score_resume`, or `ingest_resume` directly, without the calling system needing to know anything about L^AT_EX compilation, ATS scoring internals, or LLM backend selection. This makes it straightforward to fold resume-tailoring into a larger job-search agent, a career-coaching assistant, or an internal recruiting tool.

11.4 Web Application

A FastAPI backend exposes a small JSON API (POST `/api/jobs`, GET `/api/jobs/{id}` for live progress, and artifact downloads for the PDF, L^AT_EX source, cover letter, and score report) behind a React and Tailwind CSS frontend with animated progress and results views. The frontend can run against the backend either in development (Vite dev server with API proxying) or in a single-process production mode where FastAPI serves the built static assets directly.

12 Performance Observations

The figures below are measured, single-run observations captured during development and verification of this release, not a controlled statistical benchmark suite. They are reported to illustrate relative order of magnitude, not as guaranteed figures.

Scenario	Backend	Observed Time
Full pipeline, cache miss, 3 optimization iterations	Local Ollama (3B model)	~3 minutes
Full pipeline, cache miss, 3 iterations, plus cover letter	Free cloud (GLM via Puter)	~73 seconds
Single structured resume parse	Local Ollama (3B model)	~70 seconds
Single structured resume parse	Local Ollama (8B model)	~151 seconds
Single structured resume parse	Free cloud (GLM)	Seconds

Across verification runs against a realistic multi-section resume and a representative backend-engineer job posting, the system reached its default target score of 80 in a single optimization iteration in every cloud-backed run observed, with final scores in the 84–88 / 100 range.

13 Roadmap

- **Multi-resume batch mode** — tailor one master profile against a list of postings in a single invocation.
- **Additional master-resume formats** — richer DOCX table handling and direct LinkedIn export ingestion.
- **Configurable scoring weights** — allow a per-industry or per-role weighting profile for the ATS scorer.
- **Deeper MCP surface** — expose the optimize-loop feedback and per-iteration history as MCP resources, not only the final result.
- **Template gallery** — additional ATS-safe $\text{\LaTeX}$ templates beyond the current single-column default.

14 Conclusion

resume-forge demonstrates that an LLM-powered document-tailoring pipeline can be built with verifiable integrity guarantees rather than trust in prompt-following alone, and that doing so does not require sacrificing either quality of output or freedom from a specific vendor's pricing. Its layered architecture — one core pipeline exposed through four distinct interfaces, running unmodified on any of seven LLM backends spanning fully local to enterprise-cloud — is designed explicitly to be integrated into larger systems rather than used only as a standalone application. The structural no-fabrication guarantee, the deterministic and inspectable ATS scorer, and the automated one-page rendering pipeline together produce a tool whose output a candidate can trust and whose behavior a developer can audit end to end.

resume-forge — Technical White Paper — Version 1.0